https://hjfy.top/arxiv/2601.02163

EverMemOS：一种面向结构化长时程推理的自组织内存操作系统

Chuanrui Hu[1,2]*, Xingze Gao[1,2]*, Zuyi Zhou[1,2], Dannong Xu[1,2], Yi Bai[1,2], Xintong Li[1,2], Hui Zhang[1,2], Tong Li[1,2], Chong Zhang[2], Lidong Bing[2]†, Yafeng Deng[1,2]†

[1]EverMind [2]Shanda Group

{chuanrui.hu, xingze.gao, zuyi.zhou, dannong.xu, baiyi, xintong.li, zhanghui, litong02, zhangchong, lidong.bing, dengyafeng}@shanda.com

摘要

大语言模型（LLMs）正越来越多地被部署为长期交互智能体，但其有限的上下文窗口使得在长时间交互中维持连贯行为变得困难。现有的记忆系统通常存储孤立的记录并检索片段，限制了其整合不断演化的用户状态和解决冲突的能力。我们提出 **EverMemOS**，一种自组织的记忆操作系统，实现了受记忆痕迹启发的计算记忆生命周期。情景迹形成将对话流转化为 *MemCells*，以捕捉情景迹、原子事实以及时间限定的 **Foresight** 信号。语义整合将 MemCells 组织成主题性的 *MemScenes*，提炼出稳定的语义结构并更新用户画像。重构性回忆执行基于 *MemScene* 的智能体检索，以生成下游推理所需的必要且充分的上下文。在 LoCoMo 和 LongMemEval 上的实验表明，EverMemOS 在增强记忆的推理任务中达到了当前最优性能。我们进一步报告了 PersonaMem v2 的画像研究以及定性案例分析，展示了以聊天为导向的能力，如用户画像构建和 **Foresight**。代码可在 https://github.com/EverMind-AI/EverMemOS 获取。

1 引言

大语言模型（LLMs）正越来越多地被部署为长期交互智能体，而非短暂的对话工具 (Yehudai et al., 2025; Ferrag et al., 2025)。在这些情景中，智能体必须在长时间的交互过程中保持一致的人设和用户模型，同时持续整合新的约束条件。这要求记忆系统不仅能够存储过往信息，还需将经验组织成连贯且不断演进的结构，以支持长时程推理 (Wu et al., 2025; Maharana et al., 2024)。

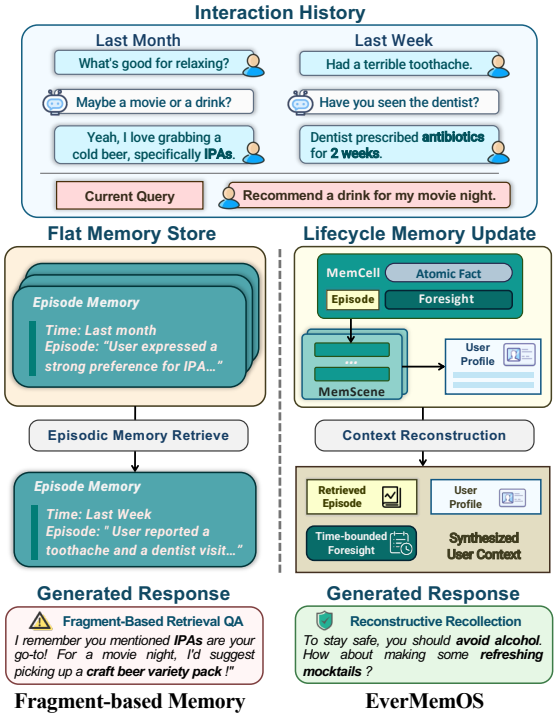

图 1: 典型基于片段的内存与 EverMemOS 在交互式聊天场景中的对比。

然而，实现这种长期一致性仍然具有挑战性。扩展上下文窗口是一种直接的方法，但超长上下文仍会导致性能下降（例如，“中间迷失”现象），并带来难以承受的计算成本 (Liu

* Equal contribution.

† Corresponding author.

et al., 2024)。更重要的是，许多失败并非源于信息缺失，而是由于整合不佳：零散的经验未能被整合为更高层次的语义结构。若缺乏整合与抽象，智能体可能检索到相关事实，却无法检测冲突、维持稳定的用户模型，或在时间上保持一致推理。

提出了多种增强记忆的方法，包括基于检索的记忆 (Zhong et al., 2024; Packer et al., 2024)、可训练的记忆模块 (Zheng et al., 2024; Gong et al., 2024)，以及最近出现的 *记忆操作系统*，它们统一了存储、检索、过滤和更新功能 (Li et al., 2025; Kang et al., 2025)。尽管这些系统提升了可扩展性和模块化程度，但大多数仍把记忆视为孤立记录的扁平集合。因此，它们缺乏显式的 *语义整合机制*——即将零散的情景经验转化为连贯、稳定的知识结构，以支持长期一致性。

我们主张，有效的长期交互需要围绕显式的生命周期来设计记忆系统，而非孤立的记忆操作。受生物记忆系统中观察到的组织原则启发，我们提出 **EverMemOS**，一个统一的记忆操作系统，将记忆建模为跨越情景迹形成、语义整合和重构回忆的动态生命周期。这种以生命周期为中心的视角超越了对单一事实的回忆，而是将经验整合为结构化的表示，从而支持随时间保持一致的推理。

图 1 展示了 EverMemOS 的设计直觉。基于片段的系统可能会回忆起用户偏好印度淡色艾尔（IPA），并推荐含酒精饮品，却未能考虑用户正在服用抗生素这一新引入的约束。相比之下，EverMemOS 将这些经验整合为对用户状态的连贯表示，使智能体能够安全地推荐无酒精替代品。尽管此类面向前瞻的行为未被现有基准明确捕捉，但它们暴露了基于片段记忆的根本局限性，并推动了 EverMemOS 的系统级设计。

如图 2 所示，EverMemOS 通过两个系统级原则来应对这些故障模式。首先，**语义整合**将情景痕迹变换为稳定的、场景级别的结构(称为 *MemScenes*)，支持连贯的聚合，例如在交互过程中保持一致的用户档案。其次，**重构性回忆**在 *必要性与充分性原则*的引导下，仅主动组合完成特定查询所必需的有根基的上下文，而非不加区分地检索所有可能相关的记录。

EverMemOS 并不旨在从神经层面模拟生物记忆。相反，它借鉴了生物记忆系统的组织原则，并将其转化为计算框架。具体而言，我们实现了将短暂的情景体验整合为稳定语义结构的过程 (McClelland et al., 1995; Frankland and Bontempi, 2005)。这种基于生命周期的设计为鲁棒的长时程交互提供了可扩展的基础，同时对特定的下游任务或领域保持中立。在本研究中，我们定量评估的重点是增强记忆的推理能力，并通过定性分析展示记忆的一致性与前瞻相关行为。

我们的贡献总结如下：

- **系统设计：**我们提出了 **EverMemOS**，一个统一的内存操作系统，重新定义了内存作为生命周期的概念，从记录的被动存储转变为经验的结构化组织。
- **语义整合：**我们提出了一种整合机制，将碎片化的对话演化为结构化的 *MemScenes*，从而实现稳定的用户建模和连贯的长期推理。
- **实验验证：**我们证明了 EverMemOS 在多个长上下文基准上实现了顶尖性能，验证了基于生命周期的内存组织的有效性。

2 相关工作

2.1 大模型中的记忆机制

上下文窗口扩展。大模型（LLMs）受到固定长度上下文窗口的限制。先前的工作通过稀疏注意力 (Beltagy et al., 2020; Zaheer et al., 2020)、循环 (Dai et al., 2019; Bulatov et al., 2022) 和长度外推法 (Chen et al., 2024, 2025) 来扩展上下文。然而，更长的上下文并不保证有效利用：中间信息丢失现象仍然存在 (Liu

et al., 2024; Bulatov et al., 2023)，这表明仅扩展上下文不足以实现持久记忆。

检索增强与参数化记忆。检索增强生成（RAG）(Lewis et al., 2020) 将记忆外化以缓解窗口限制，但其可靠性依赖于检索质量 (Ram et al., 2023)。参数化方法（如 PEFT、持续学习和模型编辑）将信息内化，却常常面临遗忘和稳定性问题 (De Lange et al., 2022)。混合方法 (Wang et al., 2023; Packer et al., 2024) 虽在一定程度上缓解了部分问题，但缺乏对持久记忆的统一组织原则。

2.2 记忆系统

早期计算记忆。早期的可微分记忆系统（例如，NTM/DNC/键值记忆）(Graves et al., 2014, 2016; Miller et al., 2016) 引入了外部记忆交互，但扩展性差，且不适合现代自回归大模型。

大语言模型智能体中的记忆。随着基于大语言模型的智能体不断发展 (Xi et al., 2023; Xia et al., 2024)，记忆系统逐渐转向持久状态的整合。近期的系统引入了情景记忆 (Wang and Chen, 2025)、语义记忆 (Shinn et al., 2024) 以及分层任务记忆 (Sun and Zeng, 2025)。然而，许多设计仍依赖于碎片化的文本单元和有限的整合机制，这可能导致长时程性能下降 (Packer et al., 2024)。

记忆操作系统与框架。近期的框架将内存管理形式化为系统级运行时。一些框架关注生命周期和容量，例如 *Nemori* 的 (Nan et al., 2025) 预测驱动更新和 *MemoryOS* 的 (Kang et al., 2025) 分层控制。另一些框架，如 *Mem0* (Chhikara et al., 2025) 和 *Zep* (Rasmussen et al., 2025)，通过知识图谱优先维护结构化事实，而 *MemOS* (Li et al., 2025) 则针对跨内存类型的统一调度。

尽管这些系统在结构化组织方面取得了进展，但它们主要关注存储最优化或事实维护。EverMemOS 通过实现一个三阶段内存生命周期而脱颖而出，将情景痕迹转化为合成的语义结构，以支持稳健的长时程推理。

3 EverMemOS

3.1 框架概述

受到生物记忆痕迹生命周期的启发 (Josselyn et al., 2015)，EverMemOS 采用三阶段工作流（图 2）: (1) **情景迹形成**将交互流编码为 *MemCells*；(2) **语义巩固**将 MemCells 组织成 *MemScenes* 并更新用户画像；以及 (3) **重构性回忆**在必要性和充分性原则下执行 *MemScene* 引导的检索。

3.2 内存原语

EverMemOS 的核心是 **MemCell**，它是连接底层数据与高层语义的原子单元。形式上，一个 MemCell c 是一个元组 $c = (E, \mathcal{F}, P, M)$，其中：

- E (**回合**): 对事件的简洁第三人称叙述，作为语义锚点。
- $\mathcal{F} = \{f_1, \ldots, f_n\}$ (**原子事实**): 从 E 推导出的离散且可验证的声明，用于高准确率匹配。
- P (**远见**): 前瞻性推理（展望；例如，计划和临时状态），用有效时间区间 $[t_{start}, t_{end}]$ 标注以支持时间意识。
- M (**元数据**): 包括时间戳和源指针的上下文定位。

该结构将记忆从静态记录（$E, \mathcal{F}$）转变为具有时间定位的表示，同时也支持**预见**（P）。

3.3 第一阶段：发作性迹形成

基于记忆痕迹概念 (Josselyn et al., 2015)，第一阶段将无界的交互历史流 $\mathcal{D} = \{d_1, \ldots, d_T\}$ 变换为离散且稳定的记忆迹（MemCells）。该过程通过三步流水线实现，从嘈杂的交互数据中提炼语义信号：

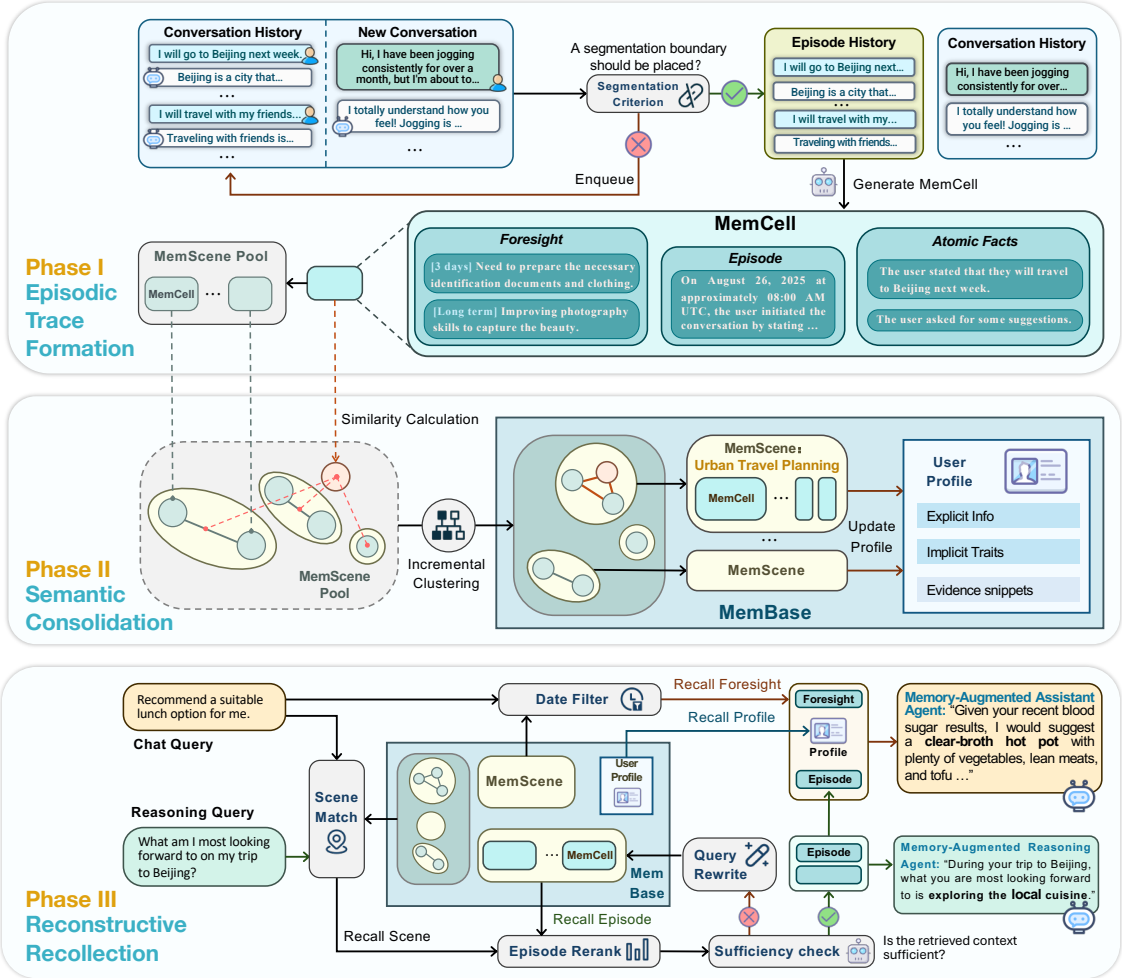

图 2: EverMemOS 工作流模拟了基于记忆痕迹的记忆生命周期：(1) **回合迹形成**将连续对话分割为包含回合、原子事实和时间限定的 **远见**的 *MemCells*。(2) **语义巩固**将 MemCells 组织成 *MemScenes* 并更新用户档案。(3) **重构性回忆**执行以 *MemScene* 为导向的检索，以构建必要且充分的上下文。

上下文分割 为了对连续流进行离散化，一个**语义边界检测器**通过滑动窗口处理交互。一旦检测到主题变化，累积的对话回合将被封装为原始的*回合历史*。我们通过大语言模型提示来实现这一步骤;尽管边界检测并非完美,但在下游评估中我们发现其具有较强的鲁棒性(表 3)。

叙事综合 为解决对话冗余和分歧问题，回合历史被综合成高保真度的 **Episode**(E)。此重写过程生成一个简洁、第三人称的叙述，解决了核心指代问题，建立了稳定的语义锚点。

结构推导 从 E 出发，系统提取 **原子事实** ($\mathcal{F}$) 用于精确匹配，并生成带有推断有效区间的**预见性**信号 (P) (例如，区分临时的“流感”与永久的“毕业”)。具体而言，我们对重写后的回合 E 进行提示，要求大语言模型输出包含有效区间约束的原子事实和预见性信号的结构化模式 $[t_{start}, t_{end}]$ 。这些组件与元数据 M 一起打包，形成最终的 MemCell c 。

3.4 第二阶段：语义整合

受系统巩固 (McGaugh, 2000) 的启发，EverMemOS 采用了一种在线机制，将 MemCells 组织成更高阶的结构，以实现从短暂回合到稳定长期知识的转移。

增量语义聚类 EverMemOS 动态组织内存。当一个新的 MemCell c 到达时，系统计算其嵌入，并检索最近的 **MemScene** 中心。如果

相似度超过阈值 τ，则 c 被同化，并增量更新场景表示；否则，将实例化一个新的 **MemScene**。此在线过程无需批量重新处理即可实时保持主题结构。

场景驱动的档案演化 场景级融合也可从聚合的证据中更新一个紧凑的**用户画像**。当新的 MemCell 被整合进 **MemScene** 时，EverMemOS 会更新简洁的场景摘要，并通过提示这些摘要（而非单个对话回合）来刷新用户画像，有助于将稳定的特质与暂时的状态分离。我们维护一个紧凑的显式事实（包括随时间变化的测量值）和隐式特质的画像，通过场景摘要进行在线更新，并采用考虑时效性的更新机制和冲突追踪（附录 B.3）。

3.5 第三阶段：重构性回忆

基于重构记忆理论 (Schacter, 2008)，EverMemOS 中的检索被建模为一种主动的 **重构** 过程，而非静态查找，其遵循 *必要性与充分性* 原则。给定一个查询 q，EverMemOS 会基于 *MemScenes* 执行 **Agentic 检索**。

MemScene 选择 我们首先通过互惠排名融合（RRF）将稠密检索和 BM25 检索在它们的原子事实 $\mathcal{F}$ 上进行融合，以计算查询与所有 MemCell 之间的相关性。然后，我们根据其组成 MemCell 中的最大相关性对每个 MemScene 进行评分，并选择得分最高的少量 MemScenes。

回合与前瞻过滤 在选定的 MemScenes 中，我们从其组成单元中汇集回合，并对其进行**重新排序**，以选择一个紧凑的集合用于下游推理。然后我们应用**前瞻性过滤**，仅保留时间有效的前瞻性 (Foresight)，其有效性区间满足 $t_{now} \in [t_{start}, t_{end}]$（丢弃过期的）。

Agentic 验证与查询重写 检索到的上下文由基于大语言模型的验证器评估其充分性。如果认为不充分，系统将触发**查询重写**步骤以补充检索；否则，将上下文传递给下游模块。提示模板见附录 C.1。

任务模式 我们考虑两种共享相同检索流水线的下游情景：**记忆增强推理**和 **记忆增强对话**。对于 *推理*，我们使用检索到的回合作为基准评估的上下文。对于 *对话*，组合的上下文还额外包含了用户档案和时间有效的前瞻信号，这些信号由当前时间 $t_{now} \in [t_{start}, t_{end}]$ 进行过滤；由于这些能力未被现有推理基准涵盖，我们通过定性案例研究进行展示。

4 实验

我们对 EverMemOS 在长时程记忆增强推理基准（LoCoMo (Maharana et al., 2024) 和 LongMemEval (Wu et al., 2025)）上进行了评估，并额外报告了 PersonaMem v2 (Jiang et al., 2025) 的性能分析研究。

4.1 实验设置

基准 我们在 LoCoMo 和 LongMemEval 上评估了增强记忆的推理能力。LoCoMo 包含 1,540 个问题，涵盖 10 个超长对话（每个对话约 ∼ 9K token），涉及单跳、多跳和时间类问题。LongMemEval（S-setting，每轮对话 ∼ 115k token）评估了 500 个问题，要求对完整历史进行解析，以检验核心能力（如更新与回避）。此外，我们还在 PersonaMem v2 上评估了用户画像能力。

基准线 我们将 EverMemOS 与最先进的内存系统进行对比：**Zep** (Rasmussen et al., 2025)，**Mem0** (Chhikara et al., 2025)，**MemOS** (Li et al., 2025)，**MemoryOS** (Kang et al., 2025)，以及 **MemU**[1]. **公平比较：**我们在方法间统一答案生成主干模型，同时保持每个基准的官方记忆配置不变；对于 LongMemEval，我们报告来自官方 **MemOS** 排行榜的基准得分。完整情景见附录 A.1。

[1]开源内存基础设施：https://github.com/NevaMind-AI/memU

评估协议 我们采用 MemOS 的 **LLM-as-a-judge** 协议：每个答案由 GPT-4o-mini 和两个辅助判断模型进行评估，并在 盲法情景下对三个判断结果的得分取平均。我们在第 A.2 节（附录）中通过与人工标注对比验证了该协议的可靠性，结果显示高度一致（Cohen's $\kappa > 0.89$）。

实现细节 EverMemOS 使用 GPT-4.1-mini（或在指定情况下使用 GPT-4o-mini）执行所有推理和记忆操作。检索采用混合稠密 +BM25 融合（RRF）并结合重排序；默认检索超参数见附录 A.1。除非另有说明，定量实验均使用**记忆增强推理**。我们在附录（表 8）中按生命周期阶段提供了 token 级别的成本分解。

4.2 主要结果

主要结果如表 1（LoCoMo）和表 2（LongMemEval）所示。我们得出三点观察。

(1) **总体而言，EverMemOS 在两个基准测试中均取得了最佳性能**。EverMemOS 在每个基准测试中均优于最强的基准——在 LoCoMo 上优于 **Zep**，在 LongMemEval 上优于 **MemOS**。

(2) **收益集中在需要整合分散证据的任务上**。在 LoCoMo 中的多跳/时间类问题上，以及在 LongMemEval 中的**知识更新** / **SS-Asst** 任务上，性能提升最为显著，而开放领域任务的表现则与 **Zep** 相当（表 1，表 2）。

(3) **EverMemOS 在准确率与效率之间提供了良好的权衡**。图 5 显示，EverMemOS 在中等检索预算下仍能实现较高的准确率，表明其性能提升源于对必要且充分的证据进行组合，而非盲目扩展上下文。

4.3 消融研究

我们对 LoCoMo 进行消融实验，以分离 *MemScenes*、*MemCells* 和回合分割的贡献。

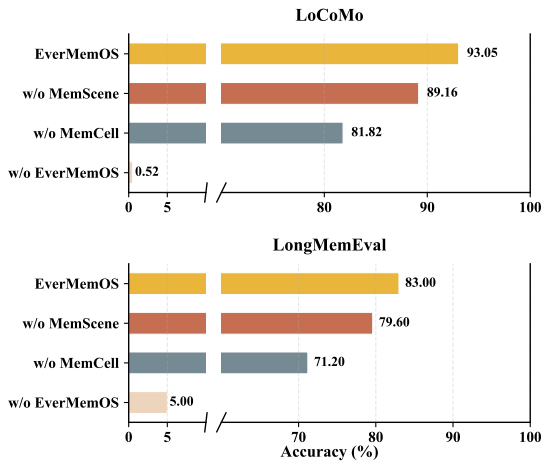

图 3: LoCoMo 和 LongMemEval 上的消融实验结果（总体准确率）

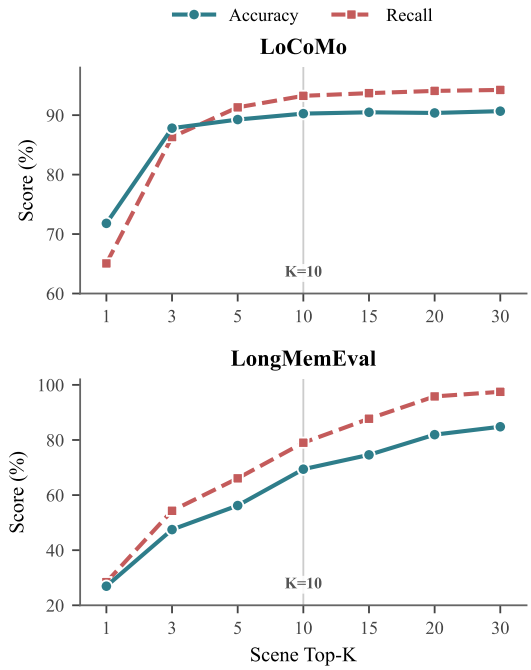

图 4: MemScene 计数的敏感性分析（N）。

4.3.1 内存架构的影响

为分离记忆结构的贡献，我们对比了 EverMemOS 与三种退化变体：无 EverMemOS（无外部记忆）、无 MemScene（在 MemCells 上进行扁平检索）以及无 MemCell（在原始对话上进行检索）。主干模型和提示保持不变，仅改变记忆表示和检索流水线。

如图 3 所示，随着结构的逐步移除，性能呈阶梯式下降，揭示了三种对应的能力损失。移除

Method	Avg. Tokens	Single Hop	Multi Hop	Temporal	Open Domain	Overall
GPT-4o-mini backbone						
MemoryOS	5.2k	62.43	56.50	37.18	40.28	54.70
Mem0	1.0k	66.71	58.16	55.45	40.62	61.00
MemU	4.0k	72.77	62.41	33.96	46.88	61.15
MemOS	2.5k	81.45	69.15	72.27	60.42	75.87
Zep	1.4k	88.11	71.99	74.45	66.67	81.06
EverMemOS	2.5k	**91.08**	**86.17**	**81.93**	**66.67**	**86.76**
GPT-4.1-mini backbone						
MemoryOS	5.5k	67.30	59.34	42.26	59.03	60.11
Mem0	1.0k	68.97	61.70	58.26	50.00	64.20
MemU	4.0k	74.91	72.34	43.61	54.17	66.67
MemOS	2.5k	85.37	79.43	75.08	64.58	80.76
Zep	1.4k	90.84	81.91	77.26	75.00	85.22
EverMemOS	2.3k	**96.67**	**91.84**	**89.72**	**76.04**	**93.05**

表 1: **LoCoMo** 在两种主干网络下的主要结果。所有指标均为准确率（%），除 Avg. Tokens 外。

Method	Token	SS-User	SS-Asst	SS-Pref	Multi-S	Know. Upd	Temp. Reas	Overall
MemU	0.5k	67.14	19.64	76.67	42.10	41.02	17.29	38.40
Zep	1.6k	92.90	75.00	53.30	47.40	74.40	54.10	63.80
Mem0	1.1k	82.86	26.78	90.00	63.15	66.67	72.18	66.40
MemOS	1.4k	95.71	67.86	**96.67**	70.67	74.26	**77.44**	77.80
EverMemOS	2.8k	**97.14**	**85.71**	93.33	**73.68**	**89.74**	**77.44**	**83.00**

表 2: **LongMemEval** 上的主要结果（准确率，%）。SS 表示单会话任务；基准结果来自官方 **MemOS** 的结果 (Li et al., 2025)。

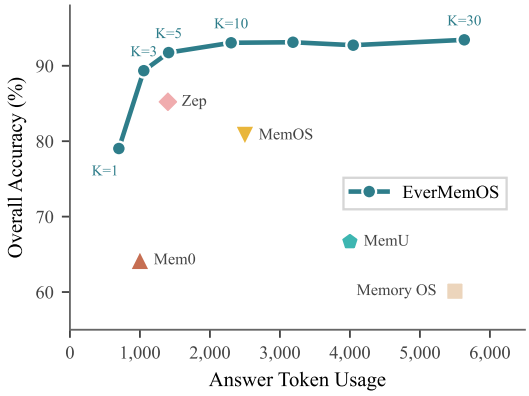

图 5: 在 LoCoMo 上通过改变检索的回合数（K）得到的性能与成本前沿。

MemScenes 会消除场景级组织，削弱对相关回合的跨回合聚合能力。进一步移除 *MemCells* 会降低稳定语义单元（回合/事实）的保持，迫使检索依赖原始对话匹配。最终，移除外存会导致长时程性能崩溃，表明许多查询无法仅依靠上下文窗口可靠处理。

4.3.2 回合分割的有效性

我们在 **w/o MemScene** 条件下，针对固定启发式方法和真实边界评估语义回合分割，以隔离边界质量的影响。

我们比较了三种策略：(1) **固定启发式**（固定的消息数量 $N = 10$ 或 token 阈值 $N = 512, 1024$）；(2) **会话（理想情况）**（真实会话边界）；和 (3) **EverMemOS**（使用不同主干网络的语义分割）。

表 3 显示，(i) 语义分割在所有情况下均优于固定的启发式方法，尤其是粗粒度 token 分块；(ii) 其性能也优于会话（Oracle），表明会话并不总是最优的检索单元；(iii) 在不同边界检测骨干网络下的结果具有鲁棒性（准确率变化不超过 ≤ 0.7 个百分点）。

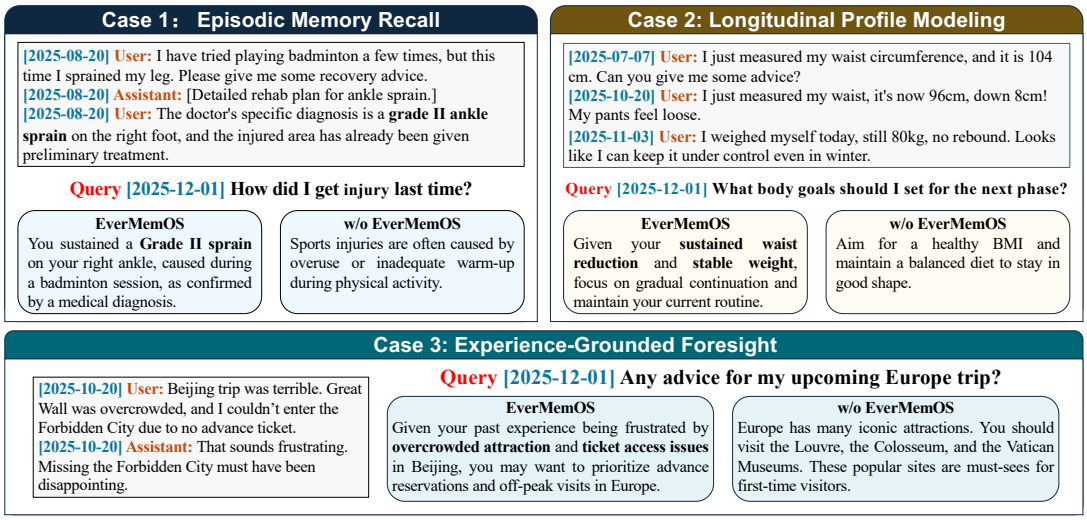

图 6: 展示记忆增强型聊天中配置文件、远见和回合功能的案例研究。

	Answer Model	
Segmentation Method	**GPT-4.1-mini**	**Qwen3-4B**
Heuristic Baselines		
Fixed-Message-10	88.05	80.95
Fixed-Token-512	87.55	80.67
Fixed-Token-1024	84.52	75.19
Semantic Segmentation		
Session (Oracle)	87.66	80.63
Default (EverMemOS)		
w/ GPT-4.1-mini	89.16	83.07
w/ Qwen3-4B	89.78	82.73

表 3: 边界检测策略的比较。**Session (Oracle)** 使用 LoCoMo 提供的真实会话划分作为地面实况。

Scenario	**Ep.+Prof.**	**Prof.-only**	**Ep.-only**
Consultation	51.03	47.33	44.44
Email (Personal)	53.85	46.15	46.15
Translation	50.00	46.15	38.08
Email (Professional)	53.79	41.38	45.17
Writing (Creative)	55.10	48.57	42.04
Writing (Professional)	45.56	44.79	40.15
Knowledge Query	63.68	62.94	54.73
Social Media	47.90	44.96	36.13
Chat	52.09	44.87	41.83
Overall	**53.25**	48.30	43.93

表 4: PersonaMem v2 上的配置文件消融实验 (Jiang et al., 2025) (跨 9 种场景的 5,000 个问题；准确率，%)

4.4 超参数分析

我们通过两个超参数来研究检索范围的影响：检索到的 *MemScenes* 数量 (N) 和回合数 (K)。如图 4 所示，性能提升在约 $N = 10$ 时趋于饱和。图 5 进一步展示了由 K 决定的效率-准确率边界。因此，我们采用 $N = 10$ 和 $K = 10$ 作为默认配置，以在性能与计算成本之间取得平衡。详细的敏感性分析见附录 B.1。

4.5 概况研究

我们评估了合并后的用户资料对 PersonaMem v2 (32k) (Jiang et al., 2025) 的影响；由于任务设置和标注的差异，不同数据集版本的结果无法直接比较。表 4显示，将用户资料加入到回合证据中，相较于仅使用回合证据，整体准确率提升了 9.32 分 (53.25 vs. 43.93)，这表明语义整合提供了超出回合检索的补充信号。关于 PersonaMem v2 上与其他记忆系统的完整对比，我们推迟至附录 A.4。

4.6 案例研究

现有的基准主要评估答案层面的准确率/召回率，未能捕捉长期会话智能体所需的若干能力，例如冲突检测、档案稳定性以及基于经验的前瞻能力。为补充定量结果，图 6 展

示了三个代表性案例：**(Episode)** 重构具体的过去事件（在羽毛球运动中发生的二级踝关节扭伤），而非生成泛化的解释；**(Profile)** 保持纵向稳定性，并利用持续改善（腰围从 104→96 cm 且体重稳定）进行轨迹一致的目标设定；以及 **(Foresight)** 借助先前观察到的失败（人群拥挤和错过提前购票）为未来出行做出主动建议。这些案例共同展示了超越现有基准测量范围的一致性、基于经验的行为。

5 结论

本文介绍了 EverMemOS，一种受记忆痕迹启发的长时程大语言模型智能体内存操作系统。通过建模显式的记忆生命周期——情景迹形成、语义巩固和重构回忆，EverMemOS 在增强记忆推理基准测试中取得了当前最优性能，尤其在多跳和时间相关问题上表现尤为突出。我们希望 EverMemOS 能为构建更一致、更具上下文感知能力的交互式智能体提供一个可扩展的基础。

局限性

我们在仅包含文本的对话基准上评估 EverMemOS。尽管 *MemCell*/*MemScene* 抽象具有模态无关性，但将 EverMemOS 扩展至多模态或具身情景，并在多模态基准上进行评估，已超出本文的研究范围。

EverMemOS 引入了由大语言模型 (LLM) 驱动的内存构建与检索操作，相较于单次遍历的基准方法，其延迟和计算成本有所增加。尽管许多组件可以被缓存、批量处理或异步运行，但提升端到端效率仍是未来的工作。

最后，当前的基准测试缺乏针对超长时序压力测试的标准协议，因此我们的评估未能完全隔离此类场景下的性能表现。这促使未来需要建立针对长期记忆组织与整合的基准测试。

References

Iz Beltagy, Matthew E Peters, and Arman Cohan. 2020. Longformer: The long-document transformer. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics.

Atilim Gunes Bulatov, Valentin Khrulkov, Leyla Mirvakhabova, Alexey Markov, Artem Babenko, and Ivan Oseledets. 2022. Recurrent memory transformer. In *Advances in Neural Information Processing Systems*, pages 20230–20243.

Aydar Bulatov, Yuri Kuratov, and Mikhail S. Burtsev. 2023. Scaling transformer to 1m tokens and beyond with rmt. *ArXiv*, abs/2304.11062.

Guanzheng Chen, Xin Li, Zaiqiao Meng, Shangsong Liang, and Lidong Wang. 2024. Clex: Continuous length extrapolation for large language models. In *International Conference on Learning Representations*.

Guanzheng Chen, Xin Li, Michael Qizhe Shieh, and Lidong Bing. 2025. LongPO: Long context self-evolution of large language models through short-to-long preference optimization. In *The Thirteenth International Conference on Learning Representations*.

Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building production-ready ai agents with scalable long-term memory. *Preprint*, arXiv:2504.19413.

Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V Le, and Ruslan Salakhutdinov. 2019. Transformer-XL: Attentive language models beyond a fixed-length context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 2978–2988. Association for Computational Linguistics.

Matthias De Lange, Rahaf Aljundi, Marc Masana, Sarah Parisot, Xu Jia, Aleš Leonardis, Greg Slabaugh, and Tinne Tuytelaars. 2022. A continual learning survey: Defying forgetting in classification tasks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(7):3366–3385.

Mohamed Amine Ferrag, Norbert Tihanyi, and Merouane Debbah. 2025. From llm reasoning to autonomous ai agents: A comprehensive review. *arXiv preprint arXiv:2504.19678*.

Paul W Frankland and Bruno Bontempi. 2005. The organization of recent and remote memories. *Nature Reviews Neuroscience*, 6(2):119–130.

Yue Gong and 1 others. 2024. M+: An efficient memory structure for large language models. *arXiv preprint arXiv:2404.09337*.

Alex Graves, Greg Wayne, and Ivo Danihelka. 2014. Neural turing machines. *arXiv preprint arXiv:1410.5401*.

Alex Graves, Greg Wayne, Malcolm Reynolds, Tim Harley, Ivo Keck, William O'Brien, Alistair Kritzman, Stanislav Illarionov, Edward Grefenstette, Tiago Wuthrich, and 1 others. 2016. Hybrid computing using a neural network with dynamic external memory. *Nature*, 538(7626):471–476.

Bowen Jiang, Yuan Yuan, Maohao Shen, Zhuoqun Hao, Zhangchen Xu, Zichen Chen, Ziyi Liu, Anvesh Rao Vijjini, Jiashu He, Hanchao Yu, Radha Poovendran, Gregory Wornell, Lyle Ungar, Dan Roth, Sihao Chen, and Camillo Jose Taylor. 2025. Personamem-v2: Towards personalized intelligence via learning implicit user personas and agentic memory. *arXiv preprint arXiv:2512.06688*.

Sheena A Josselyn, Stefan Köhler, and Paul W Frankland. 2015. Finding the engram. *Nature Reviews Neuroscience*, 16(9):521–534.

Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. 2025. Memory os of ai agent. *arXiv preprint arXiv:2506.06326*.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Yuxiang Kukliansky, Wen-tau Yih Chen, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474.

Zhiyu Li, Shichao Song, Chenyang Xi, Hanyu Wang, Chen Tang, Simin Niu, Ding Chen, Jiawei Yang, Chunyu Li, Qingchen Yu, and 1 others. 2025. Memos: A memory os for ai system. *arXiv preprint arXiv:2507.03724*.

Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.

Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. Evaluating very long-term conversational memory of llm agents. *Preprint*, arXiv:2402.17753.

James L McClelland, Bruce L McNaughton, and Randall C O'Reilly. 1995. Why there are complementary learning systems in the hippocampus and neocortex: insights from the successes and failures of connectionist models of learning and memory. *Psychological review*, 102(3):419.

James L McGaugh. 2000. Memory–a century of consolidation. *Science*, 287(5451):248–251.

Alexander Miller, Adam Fisch, Jesse Dodge, Amir-Hossein Karimi, Antoine Bordes, and Jason Weston. 2016. Key-value memory networks for directly reading documents. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 1400–1409. Association for Computational Linguistics.

Jiayan Nan, Wenquan Ma, Wenlong Wu, and Yize Chen. 2025. Nemori: Self-organizing agent memory inspired by cognitive science. *Preprint*, arXiv:2508.03341.

Charles Packer, Vivian Woodside, Neal Dhir, and Douwe Kiela. 2024. Memgpt: Towards llms as operating systems. In *Advances in Neural Information Processing Systems*.

Ori Ram, Eyal Shnarch, Jonathan Uziel, Lisa Haklay, and Amir Globerson. 2023. In-context retrieval-augmented language models. *Transactions of the Association for Computational Linguistics*, 11:1316–1331.

Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. 2025. Zep: A temporal knowledge graph architecture for agent memory. *Preprint*, arXiv:2501.13956.

Daniel L Schacter. 2008. *Searching for memory: The brain, the mind, and the past*. Basic books.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2024. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36.

Haoran Sun and Shaoning Zeng. 2025. Hierarchical memory for high-efficiency long-term reasoning in llm agents. *arXiv preprint arXiv:2507.22925*.

Weizhi Wang, Li Dong, Hao Cheng, Xiaodong Liu, Xifeng Yan, Jianfeng Gao, and Furu Wei. 2023. Longmem: Augmenting language models with long-term memory. In *Advances in Neural Information Processing Systems*, volume 36, pages 20292–20306.

Yu Wang and Xi Chen. 2025. Mirix: Multi-agent memory system for llm-based agents. *arXiv preprint arXiv:2507.07957*.

Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2025. Longmemeval: Benchmarking chat assistants on long-term interactive memory. *Preprint*, arXiv:2410.10813.

Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yi Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, and 1 others. 2023. The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864*.

Chun Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2024. Agentless: Demystifying llm-based software engineering agents. *ArXiv*, abs/2407.01489.

Asaf Yehudai, Lilach Eden, Alan Li, Guy Uziel, Yilun Zhao, Roy Bar-Haim, Arman Cohan, and Michal Shmueli-Scheuer. 2025. Survey on evaluation of llm-based agents. *arXiv preprint arXiv:2503.16416*.

Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontañón, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. 2020. Big Bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, pages 17283–17296.

Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. 2025. Qwen3 embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176*.

Yuan Zheng and 1 others. 2024. Memoryllm: A framework for personalized and long-term dialogue generation. *arXiv preprint arXiv:2401.17122*.

W Zhong, L Guo, Q Gao, and 1 others. 2024. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19724–19731.

A 评估细节

A.1 评估情景与公平比较

LoCoMo 骨干网络。 我们报告了在两种主干网络下的 LoCoMo 结果：GPT-4.1-mini（主要，反映最新的主干网络）和 GPT-4o-mini（便于与先前工作进行比较）。遵循 LTMOS 评估中的常见做法，我们将用于最终答案生成的主干网络标准化，以隔离记忆管理对基础模型的贡献。

基准可执行性。 对于 EverMemOS 和 MemoryOS，我们使用指定的主干网络执行完整的流水线（内存构建、检索和回答）。对于 Mem0、MemU、MemOS 和 Zep，我们使用其官方 API 进行内存管理/检索；在此情景下，我们保留每个基准模型的官方内存配置和提示不变，仅在回答阶段应用统一的主干网络。

LongMemEval。 由于 LongMemEval 的输入长度极长，我们无法稳定地端到端运行所有基准模型 API；因此，我们报告来自官方 MemOS 排行榜的基准结果。[2] 并使用相同协议对 EverMemOS 与 GPT-4.1-mini 进行评估。

检索配置。 EverMemOS 使用一种混合检索器，通过倒数排名融合（RRF）将稠密检索（编码器：Qwen3-Embedding-4B (Zhang et al., 2025)）与稀疏检索（BM25）相结合，随后进行回合重排序（Qwen3-Reranker-4B（Zhang et al., 2025））。除非另有说明，我们检索前 10 个 *MemScenes* 并为下游推理选择 10 个回合。

MemBase 的统计量和构建超参数。 我们报告了 LoCoMo 和 LongMemEval 的数据集级别 MemBase 统计量（表 5）以及内存构建超参数（表 6）。*MemScenes* 是第二阶段生成的聚类单元，每个 *MemScene* 包含一组少量的 *MemCells*。我们在不同数据集上使用相同的流水线，同时采用数据集特定的聚类超参数以反映不同的对话结构和时间跨度。LongMemEval 包含 500 个对话-问题对（每条对话对应一个）。**最大时间间隔**是允许的最大时间距离（以天为单位）：在将一个 *MemCell A* 分配给候选 *MemScene* 时，如果该 *MemScene* 中最近时间点的 *MemCell B* 的时间距离超过此阈值，则 *A* 不会被聚类到该 *MemScene*。

[2]排行榜数据：https://huggingface.co/datasets/MemTensor/MemOS_eval_result

表 5: MemBase 在 LoCoMo 和 LongMemEval 上的统计量。

Metric	LoCoMo	LongMemEval
Dataset scale		
#Conversations	10	500
#Questions	1,540	500
MemBase statistics		
#Total MemCells	702	54,755
#Total MemScenes	286	40,138
Avg MemCells/conv.	70.2	109.5
Avg MemScenes/conv.	28.6	80.3
Avg MemCells/MemScene	2.45	1.36
MemCells/conv. (range)	34–95	82–154
MemScenes/conv. (range)	13–49	60–102

表 6: 内存构建超参数。

Hyperparameter	LoCoMo	LongMemEval
Clustering threshold τ	0.70	0.50
Max time gap (days)	7	30

多轮查询重写频率。 在 LoCoMo（GPT-4.1-mini）上，充分性检查器会触发 31.0% 的问题的第二轮查询重写。

默认评估模式。 除非另有说明，定量实验均采用**记忆增强推理**（仅回合）。我们还在表 4中额外报告了整合后的**档案**的影响，而**预见性**则在定性的**案例研究**（**记忆增强聊天**）中予以展示。

A.2 大语言模型作为评判者的一致性

我们从 LoCoMo 中随机选取了 25 个不重叠的问答对，从 LongMemEval 中也选取了 25 个，为每个问题生成了模型答案。我们通过 Prolific 招募了标注者。对于每一对问答，五名

独立的人类评估者根据问题和参考答案判断生成的答案是否正确。所有参与者均通过平台界面提供了知情同意，并获得了约每小时 12.00 美元的报酬，符合学术研究的公平薪酬指导原则，并高于当地最低工资标准。表 7 显示，基于大模型作为评判者的协议与人工标注之间具有高度一致性：Cohen' s κ 超过 0.89，且在各基准上的准确率均保持在 98% 以上。在 LoCoMo 上的 Pearson r 为 0.891，在 LongMemEval 上为 0.979。这些结果表明，GPT-4o-mini 在答案验证任务中达到了人类水平的可靠性，从而实现了严谨、可复现且成本高效的评估。

表 7: 大语言模型作为裁判的可靠性矩阵。

Model	Cohen's κ	95% CI	Accuracy	Pearson r
LoCoMo	0.891	[0.742, 1.000]	0.984	0.891
LongMemEval	0.978	[0.936, 1.000]	0.992	0.979

A.3 Token 成本明细

为提升成本透明度，我们在 LoCoMo 评估（1,540 个问题）过程中记录了所有 LLM API 调用，并将 token 使用量归因于流水线中的各个阶段。由于 LoCoMo 评估采用 **Memory-Augmented Reasoning**（仅回合模式），我们并未调用 **Profile** 模块；因此，与 profile 相关的 token 不包含在表 8 中。表 8 将各阶段映射至 EverMemOS 的阶段。阶段 I 对应记忆构建（add）。在此仅回合设置下，阶段 II 使用非 LLM 计算（聚类/嵌入更新），因而不产生额外的 LLM token。阶段 III 包含检索（search）和答案生成（answer）。evaluate 阶段反映 LLM 作为评判者进行评分（每个问题三位评判者），并单独报告。使用 GPT-4.1-mini 时，阶段 III 消耗 10.27M token（$\sim$ 6.7k/回合）；使用 GPT-4o-mini 时，消耗 9.31M token（$\sim$ 6.0k/回合）；阶段 I 分别消耗 9.42M 和 9.34M token，按记忆构建过程分摊。

A.4 PersonaMem v2：完整对比结果

表 9 报告了在 PersonaMem v2 (32k) (Jiang et al., 2025)（跨 9 种场景的 2,447 个问题）上的完整对比结果。其中 *Profile* 行表示记忆系统是否提供类似 "个人资料" 的组件（不一定是名为 "Profile"），该组件用于总结稳定的用户信息（例如，MemOS 维护显式与隐式偏好）。对于具有此类组件的方法（✓），我们使用检索到的记忆 加上系统的类似个人资料组件生成答案；对于不具有该组件的方法（✗），仅使用检索到的记忆生成答案。EverMemOS 达到了最高的整体准确率（53.25%），比最强基准（MemOS，50.72%）高出 2.53 个百分点。

表 8: LoCoMo（1,540 个问题）在两种骨干模型下的 token 级成本分解。token 以百万（M）为单位报告；总计包括提示和生成部分。

Stage	#Calls	Prompt (M)	Total (M)
GPT-4.1-mini			
add	7056	8.66	9.42
search	2017	4.12	4.45
answer	1540	4.63	5.82
search+answer	**3557**	**8.75**	**10.27**
evaluate	4620	2.35	2.38
GPT-4o-mini			
add	7250	8.60	9.34
search	2219	4.37	4.62
answer	1540	3.84	4.69
search+answer	**3759**	**8.21**	**9.31**
evaluate	4620	2.14	2.17

B 附加分析

B.1 超参数敏感性与效率权衡

为了更好地理解检索预算，我们在一个简化的情景下分析了 MemScene 预算 N 和回合预算 K，该情景禁用了第三阶段的 Agentic 验证与重写环，从而隔离了 one-shot 检索。图 4 显示，增加 N 会初期提升证据会话召回率和答案准确率，但很快达到饱和；N=10 已经能够实现较强的召回率。因此，我们避免通过暴力扩展检索到的场景集合以提高效率。我们还将 N=10 设为确保候选池中即使在极端情况下（每个检索到的 MemScene 仅包含一个 MemCell）也至少包含 K=10 个 MemCells。我们选

Scenario	Zep	Mem0	MemU	MemoryOS	MemOS	EverMemOS
Consultation	39.51	43.21	37.86	35.80	48.15	**51.03**
Email (Personal)	42.51	41.30	33.20	36.84	49.80	**53.85**
Translation	36.92	43.08	38.46	40.00	51.92	**50.00**
Email (Professional)	37.59	42.41	32.76	35.86	50.00	**53.79**
Creative Writing	41.22	42.86	35.51	35.51	48.16	**55.10**
Writing (Professional)	40.54	34.75	35.14	35.91	**48.26**	45.56
Knowledge Query	63.43	59.20	56.97	57.96	61.94	**63.68**
Social Media	32.35	38.66	34.03	35.29	46.64	**47.90**
Chat	44.87	40.30	34.22	36.88	44.87	**52.09**
Profile	✗	✗	✓	✓	✓	✓
Overall	43.40	43.85	38.70	40.05	50.72	**53.25**

表 9: PersonaMem v2 (32k) 与 (Jiang et al., 2025)（跨 9 种场景，共 5,000 道问题；准确率，%）的全面对比

择 K=10 个回合，因为大多数记忆问题可以通过一组紧凑的回合解答，同时仍能覆盖那些标注证据跨度达 7–8 个召回回合的困难实例。最后，图 5 展示了有利的成本-准确率前沿：降低 K 可显著减少下游推理所用 token 数量，而在中等 K 值下，EverMemOS 能够在更低的 token 使用量和更高的准确率上优于强基准。

B.2 LoCoMo 上的准确率超过召回率

在图 4 中，当 LoCoMo 的 K 较小时，准确率可以超过召回率。表 10 量化了这一现象：即使未检索到任何 * 标注 * 的证据会话（“零召回”），仍有 12–20% 的问题能够被正确回答。

表 10: 准确率与召回率统计结果对比（LoCoMo）。

Metric	K=1	K=3
Recall	65.06%	86.32%
Accuracy	71.80%	87.81%
Zero-recall questions	429	125
Answered correctly	52 (12.1%)	25 (20.0%)

这主要反映了**信息冗余**和**非唯一证据标注**：显著事实（身份、偏好、目标）在多个会话中重复出现，因此标注的证据并不总是支持答案的唯一会话。例如，关于“Caroline 的身份”的问题被标注为会话 [1]，但会话 [11–15] 也提到她是跨性别女性，使得从其他会话中也能得出正确答案。此外，当确切标注的会话缺失时，大模型仍可能通过语义相关的检索内容推断出正确回答。

总体而言，当证据分布时，基于标注证据计算的召回率可能会低估检索的有用性。将 K 从 1 增加到 3 可使零召回情况减少 71%(429→125)，缩小准确率与召回率之间的差距。

示例案例。 我们提供三个具有代表性的例子，这些例子中即使缺少标注的证据片段，答案仍然正确。

- **冗余的身份事实。** *Q: “Caroline 的身份是什么？”* 标准答案是 *跨性别女性*。尽管证据在会话 [1] 中已标注，但后续会话也明确提到了这一身份；检索器在小 K 时展示了这些替代信息，模型因此正确作答。

- **分布式活动提及。** *Q: “梅兰妮参与哪些活动？”* 标准答案跨越多个爱好（例如，陶艺、露营、绘画、游泳），证据在多个会话中被标注。检索到的会话可能遗漏了被标注的会话，但仍包含足够的提及内容（例如，陶艺/绘画）以支持正确回答。

- **从相关信号推断。** *Q: “Caroline 会将写作作为职业选择吗？”* 尽管证据在会话 [7] 中进行了标注，但其他会话中检索到的

内容描述了她的职业目标（例如成为咨询师），这使得 LLM 能够推断出写作不太可能是她的职业选择。

B.3 资料提取示例

EverMemOS 维护一个紧凑的 **用户档案**，包含两个字段：*显式事实*（可验证的属性和随时间变化的测量值）和 *隐式特质*（偏好和习惯）。该档案会从第二阶段场景摘要中在线更新，对随时间变化的字段采用考虑时效性的更新方式，并在证据不一致时进行冲突追踪。表 11 提供了一个简化的示例。

C 可复现性成果

C.1 Agentic 检索提示

为了使我们的系统行为透明且可复现，我们包含了智能体检索控制器所使用的核心提示模板。[3]

充分性检查。 我们使用基于大语言模型的充分性检查来判断当前检索到的文档是否包含回答用户查询所需的足够证据。提示模板（含占位符）如下所示。

```
You are an expert in information retrieval evaluation.
↪ Assess whether the retrieved documents provide
↪ sufficient information to answer the user's query.

User Query:
{query}

Retrieved Documents:
{retrieved_docs}

### Instructions:

1. **Analyze the Query's Needs**
  - **Entities**: Who/What is being asked about?
  - **Attributes**: What specific details (color, time,
  ↪ location, quantity)?
  - **Time**: Does it ask for a specific time (absolute
  ↪ or relative like "last week")?

2. **Evaluate Document Evidence**
  - Check **Content**: Do the documents mention
  ↪ the entities and attributes?
  - Check **Dates**:
    - Use the `Date` field of each document.
    - For relative time queries (e.g., "last week",
    ↪ "yesterday"), verify if document dates fall
    ↪ within that timeframe.
    - If the query asks "When did X happen?", do you
    ↪ have the specific date or just a vague
    ↪ mention?

3. **Judgment Logic**
  - **Sufficient**: You can answer the query
  ↪ *completely* and *precisely* using ONLY the
  ↪ provided documents.
  - **Insufficient**:
    - The specific entity is not found.
    - The entity is found, but the specific attribute
    ↪ (e.g., "price") is missing.
    - The time reference cannot be resolved (e.g., doc
    ↪ says "yesterday" but has no date, or doc
    ↪ date doesn't match query timeframe).
    - Conflicting information without resolution.

### Output Format (strict JSON):
{{
  "is_sufficient": true or false,
  "reasoning": "Brief explanation. If insufficient, state
  ↪ WHY (e.g., 'Found X but missing date', 'No
  ↪ mention of Y').",
  "key_information_found": ["Fact 1 (Source: Doc
  ↪ 1)", "Fact 2 (Source: Doc 2)"],
  "missing_information": ["Specific gap 1", "Specific
  ↪ gap 2"]
}}

Now evaluate:
```

多查询生成（压缩版）。 当当前检索被认为不充分时，我们生成 2-3 个补充的后续查询，以获取缺失的信息。我们省略示例，仅保留影响行为的约束条件（输入、策略选择以及严格的 JSON 输出格式）。

```
You are an expert at query reformulation for
↪ conversational memory retrieval.
Your goal is to generate 2-3 complementary queries to
↪ find the MISSING information.

--------------------------
Original Query:
{original_query}
```

[3] 我们的实现已开源；我们仍在此处包含提示内容，以确保论文在评审时具有自洽性。

表 11: 示例（去标识化）：摘要化的证据片段及生成的用户档案。

Evidence snippets (excerpt)	**Retrieved user profile (excerpt)**
2025-07-07: "I just measured my waist circumference, and it is 104 cm. Can you give me some advice?" 2025-10-20: "My waist is now 96 cm, down 8 cm! My pants feel loose." 2025-11-03: "The doctor said my fatty liver has improved (moderate → mild). Waist is now 95 cm." 2025-11-03: "My weight is still 80 kg, no rebound. I can keep it under control even in winter."	**Explicit facts.** Waist circumference: baseline 104 cm; latest 95 cm ($\Delta = -9$ cm). Weight: stable at 80 kg (no rebound). Fatty liver grade: moderate → mild (improved). **Implicit traits.** Self-management: goal-oriented; consistently tracks health metrics and responds well to feedback. Preference: requests immediately actionable adjustments.

```
Key Information Found:
{key_info}

Missing Information:
{missing_info}

Retrieved Documents (Context):
{retrieved_docs}
--------------------------

### Strategy Selection (choose based on why info is missing)
- Pivot / Entity Association: search related entities/categories
- Temporal Calculation: anchor relative times using document dates
- Concept Expansion: synonyms / general-specific variants
- Constraint Relaxation: remove one constraint at a time

### Query Style Requirements (use DIFFERENT styles)
1) Keyword Combo (2-5 words)
2) Natural Question (5-10 words)
3) Hypothetical Statement (HyDE, 5-10 words)

### Output Format (STRICT JSON)
{
  "queries": ["Query 1", "Query 2", "Query 3"],
  "reasoning": "Strategy used for each query (e.g., Q1: Pivot, Q2: Temporal)"
}
```

C.2 端到端推理迹（LoCoMo 多跳示例）

为了提高透明度，我们为一个典型的 LoCoMo 多跳问题（对话 locomo_6）提供了端到端的推理迹，包括 MemBase 层次结构（*MemScenes* 和 *MemCells*）以及两轮检索过程（充分性检查和查询重写），这些过程最终导向了正确的答案。我们用 N 表示检索到的 MemScene 数量，用 K 表示检索到的 MemCell（回合）数量（对应于我们实现中的 scene_top_k 和 response_top_k）。

一瞥即见其迹。

- **问题（多跳）.** *"James 是否住在康涅狄格州？"* 对话中从未直接说明 James 的居住地；系统必须从相关证据中推断出答案。
- **MemBase 层次结构。** 49 个 MemScene / 91 个 MemCell；检索选择前 N=10 个 MemScene（20%），然后对 K=10 个 MemCell 进行重排序/选择以回答问题。
- **第一轮检索 + 充分性。** 前 N=10 MemScenes（31 个 MemCells）→ 不充分（is_sufficient=false）；缺少明确的居住地提及/未确认居住在康涅狄格州。
- **查询重写。** 控制器生成针对居住地/位置信息的优化查询。
- **第二轮检索。** 在 40 个额外候选单元中，排名最高的 MemCell 包含了关键证据，表明詹姆斯从斯坦福德的一家收容所领养了一只狗，从而支持了基于证据的推理。

表 12: 端到端推理迹（LoCoMo 多跳示例），概要。

Stage	Key outputs
Input	Query: *Does James live in Connecticut?* (Category: multi-hop; Gold: *Likely yes*).
MemBase	49 MemScenes / 91 MemCells (conversation locomo_6).
Round 1	Top N=10 MemScenes (31 MemCells) → insufficient (is_sufficient=false); missing an explicit residence mention / confirmation of Connecticut.
Rewrite	Refined queries: (i) James residence Connecticut; (ii) Where does James currently live; (iii) James lives near McGee's bar in Connecticut.
Round 2	+40 candidates; top result is *James Adopts Shelter Dog Ned... (Apr 12, 2022)* from cluster_004, mentioning "Stamford".
Answer	Output: *Likely yes*; judged correct by 3/3 LLM judges.

- **推理 + 评估**。最终答案：很可能是; 3 位 LLM 评委均判定为正确。

示例（格式化）。为提高可读性，我们将迹总结在表 12 中（而非打印原始 JSON）。

详细迹。 第一轮：初始检索与充分性检查。

- **检索模式**。使用 N=10 和 K=10 的 Agentic MemScene 引导重排序 (agentic_scene_rerank)。
- **检索到的候选项**。 N=10 MemScenes (31 个 MemCell)。
- **充分性判断**。 is_sufficient=false.
- **找到的关键信息**。"詹姆斯和萨曼莎最近搬到了麦基酒吧附近同居"；"詹姆斯最近去了努克".
- **信息缺失**。 (i) 詹姆斯居住地的明确提及；(ii) 确认詹姆斯是否住在康涅狄格州。

验证者推理（摘录）。

None of the documents explicitly mention where
↪ James currently lives or whether he lives in
↪ Connecticut.

查询重写（第二轮查询）。

1) James residence Connecticut
2) Where does James currently live
3) James lives near McGee's bar in Connecticut

第二轮：检索到的顶级 MemCell。经过优化的查询检索到 40 个额外候选单元；排名靠前的结果为：

Rank	Scene	Subject (summary)
1	4	**James Adopts Shelter Dog Ned and Discusses Gaming Progress with John (Apr 12, 2022)**
2	43	James Announces Move-In with Samantha Near McGee's Bar (Oct 31, 2022)
3	15	James Shares Recent Life Challenges and Support System (Jun 16, 2022)
4	44	James's Family Road Trip Visiting Friends Josh and Mark (Nov 7, 2022)
5	22	James's Upcoming Summer Trip to Toronto and Vancouver (Jul 9, 2022)

关键证据（对话摘录）。

I adopted a pup from a shelter in Stamford last week
↪ and my days have been so much happier with
↪ him in the fam.

关键证据（原子事实）

- 詹姆斯上周（2022 年 3 月 28 日至 4 月 3 日）在斯坦福的一家收容所领养了一只名叫内德的小狗。

答案 + 评价。

James adopted his dog Ned from a shelter located in
↪ Stamford, which is a city in Connecticut,
↪ strongly implying he lives near Stamford,
↪ Connecticut. Furthermore, when James and
↪ Samantha moved in together on October 31,
↪ 2022, they rented an apartment near McGee's
↪ bar, a location they both favored. Although the
↪ exact city of this apartment is not explicitly
↪ stated, given no mention of relocation to another
↪ state and the previous connection to Stamford, it
↪ is highly likely that their residence is still in
↪ Connecticut, near Stamford. Therefore, based on
↪ these details, James does live in Connecticut
↪ with high confidence.

结果：is_correct=true（3/3 位评审）。